Architecture Rules

- This project follows a Feature-First architecture.
- Business logic must remain inside the domain layer of each feature.
- UI logic and state must be handled by controllers in the presentation layer.
- Infrastructure concerns must remain inside core/services.
- Shared UI components must be placed in the shared folder.
- Controllers should aways be either a ChangeNotifier or a ValueListenable!
- Services should be SINGLETON pattern and not use de GetIt package!

```
Architecture Rules

lib/

main.dart
  -> Application entry point. Initializes AppInjector.init()
and calls runApp() with app_bootstrap.

app/
  -> Global application initialization (routing, dependency i
njection, and themes).

  app.dart
    -> Creates the MaterialApp and applies global themes and
routes.

  app_routes.dart
    -> Centralized registry of named routes.

  app_injector.dart
```

```
    -> Dependency injection configuration (singletons, servic
es, repositories).

  app_bootstrap.dart
    -> Initializes global providers and starts the applicatio
n by calling app.dart.

  providers/
    -> Global providers shared across the entire application.

  gates/
    -> Application flow gates that determine which path the a
pp should follow.

    gate_splash.dart
      -> Initial gate responsible for determining which featu
re or gate
         should be opened based on the current application st
ate.

core/
  -> Global resources that are independent of any specific fe
ature.

  entities/
    -> Entities shared across multiple features (optional).

  errors/
    -> Global error and failure classes.

  utils/
    -> Global helper functions and utilities.

  services/
    -> Global infrastructure services such as API clients, SQ
Lite,
```

```
       SharedPreferences, notifications, and other platform s
ervices.

    Rules:
      - Services MUST NOT expose DTOs outside the service lay
er.
      - Services are divided into two types:

        Technical services
        -> Provide low-level CRUD operations or infrastructur
e access.

        Orchestrator services
        -> Coordinate multiple technical services to perform
a task
           and return a DTO result.

  constants/
    -> Fixed values used throughout the application
       (strings, URLs, sizes, keys).

features/
  -> Each application feature is isolated and self-contained.

  <feature_name>/

    domain/
      -> Business rules of the feature:
         - entities
         - use cases
         - abstract repository contracts

    data/
      -> Data layer implementation:
         - repository implementations
         - DTOs
```

```
          - mappers
          - data sources (API, database, cache)

    presentation/
      -> User interface layer of the feature.

      pages/
        -> Full screens of the feature.

      widgets/
        -> UI components used only within the feature.

      controllers/
        -> Screen state controllers and UI logic.

           This layer contains:
           - Provider state controllers
           - ChangeNotifiers
           - ValueNotifiers
           - Screen-specific UI state logic

           Controllers are responsible for managing UI state,
           coordinating use cases, and notifying the UI when
state changes.
       states/
                -> Data structures representing the UI state
of a screen.

                   Responsibilities:
                   - Define all UI state fields
                   - Represent loading states
                   - Represent error states
                   - Represent current data displayed on the
screen

                   States should be simple data objects.
```

```
shared/
  -> Reusable UI resources shared across multiple features.

  widgets/
    -> Reusable widgets such as buttons, inputs, cards, and l
ayouts.

  theme/
    -> Global theme configuration including colors, typograph
y,
       spacing, and styles.

  extensions/
    -> Extensions for common types such as:
       String, DateTime, BuildContext, and Widget.

  mixins/
    -> Reusable behaviors such as:
       loading handling,
       snackbar helpers,
       lifecycle helpers,
       animations,
       validators.
```

Example file of how a Singleton Service flow must be

- Notice that there is the using of “assert()”! If there are a lot of equal ocorrencies, create a file that contains the strings of even the assert itself.
- The main thing from these Singleton services are → Using the “static” nullable variable for the instance!

- For these architecture, if needed, it can create two types of Services as mentioned before:

  Technical services
  → Provide low-level CRUD operations or infrastructure access.

  Orchestrator services
  → Coordinate multiple technical services to perform a task and return a DTO result.

```
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutterlabs/firebase_options.dart';

/// this is the Auth Service for the firebase project
class FirebaseAuthService {
  static FirebaseAuthService? _instance;
  late final FirebaseAuth _auth;

  FirebaseAuthService._();

  static FirebaseAuthService get instance {
    assert(_instance != null, 'FirebaseAuthService instance n
ot initialized!');
    return _instance!;
  }

  /// class starter
  static Future<FirebaseAuthService> init() async {
    if (_instance == null) {
      _instance = FirebaseAuthService._();
      await Firebase.initializeApp(
        options: DefaultFirebaseOptions.currentPlatform,
      );
      print('inicializou!');
      _instance!._auth = FirebaseAuth.instance;
```

```
    }



    return _instance!;
  }

  /// class 'disposer', there are two main things here:
  /// 1. Make sure that the _instance still EXISTS before mak
ing it null
  /// 2. Make sure that the _auth.currentUser is NULL!
  /// There are two asserts that prevents the things above ha
ppens during development
  static Future<void> dispose() async{
    assert(_instance != null, 'FirebaseAuthService do not exi
sts');
    assert( _instance?._auth.currentUser == null, 'User is st
ill logged! Make sure that the logout method is called proper
ly!');
    _instance = null;
  }

  /// state changes method
  Stream<User?> authStateChanges() {
    return _auth.authStateChanges();
  }



  /// create user method
  createUser(String email, String password) async{
    await _auth.createUserWithEmailAndPassword(email: email,
password: password);
  }

  /// login method
```

```
  Future<UserCredential>login(String email, String password)
async{
    return await _auth.signInWithEmailAndPassword(email: emai
l, password: password);
  }

  /// logout method
  logout() async{
    await _auth.signOut();
  }

  /// get UID method
  String? getCurrentUserUID(){
    return _auth.currentUser?.uid;
  }

  /// isAuthenticated checker
  bool isAuthenticated() {
    return _auth.currentUser != null;
  }




}
```